QAOA FOR COLUMN GENERATION

Smayan Nandakumar
Rensselaer Polytechnic Institute
Troy, NY, 12182, USA
nandas2@rpi.edu

1 INTRODUCTION

Computing high quality solutions to the Bin Packing / Cutting Stock problem is a cornerstone of combinatorial optimization with massive industrial implications. The problem that cloud / super-computing centers face of distributing computing power to as many users while minimizing compute resources, is an instance. As is the problem of delivering products to consumers, where we seek to distribute all of the product, while minimizing the number of trucks needed.

Unfortunately, this problem is NP-Complete, making this computation a challenge.

In this report, we study a method to solve Bin Packing, that involves column generation. The subproblems created by this generation are all instances of knapsack, for which we employ a few strategies.

1. A classical $\frac{1}{2}$-approximation algorithm
2. QAOA - perfect simulator
3. QAOA - noisy simulator
4. QAOA - real quantum computer

2 RELATED WORK

The paper [8] explores theoretical limits and bounds on the approximability of the Bin Packing Problem in different paradigms. For instance, they show that any online algorithm for the problem cannot approximate this problem with a factor better than $\frac{3}{2}$. The foundational paper [6] introduces the method of applying column generation to the Bin Packing problem. It utilizes efficient heuristics to solve the problem, using the standard exponential dynamic programming formulation when necessary. In this report, we consider replacing those heuristics with QAOA by formulating the subproblems as QUBOs. In order to overcome the challenges imposed by [8], we use the contributions of [6], combined with QAOA.

The specific Ansatz used to solve the subproblem is introduced in [7]. While they apply this method strictly to solve isolated instances of the Knapsack problem, we extend their work by embedding this formulation as a dynamic subroutine to iteratively generate columns for the more sophisticated Bin Packing problem.

In our report, we run circuits on real quantum hardware. However, this introduces some challenges, such as mapping qubits onto the hardware topology, while minimizing the number of swaps (ecr gates) needed. The paper [2] addresses this, and presents some solutions. In our study, we evaluate how these mappings on the real hardware affect the quality of outputs by evaluating instances of Bin Packing via column generation.

The paper [3] discusses utilizing a hybrid quantum-classical column generation approach to difficult combinatorial problems, similar to our algorithm. They have detailed results on noisy and noiseless simulation of their methods, whereas we also run our algorithm on real quantum hardware.

3 Problem Formulation

In the 0/1 Bin Packing/ Cutting Stock Problem, we are given a positive integer W, a set of items I, and an associated weight per item: $w_i, \forall i \in I$.
The Bin Packing problem is defined as the following Integer Linear Program (from [1]):

$$\begin{aligned} \min \quad & \sum_{j=1}^{n} x_j \\ \text{subject to} \quad & \sum_{j=1}^{n} a_{ij} x_j = 1, \quad i = 1, \ldots, m \\ & x_j \in \{0, 1\}, \qquad j = 1, \ldots, n \end{aligned} \tag{BP}$$

Here, each column of A, denoted A_i, is a valid pattern, which is a vector of 0s and 1s, such that

$$\sum_{a_{ij} \in A_i} w_i a_i j \leq W$$

Thus, if A contained every valid such pattern, then the above lp perfectly models the problem of choosing a collection of patterns that together pack all items, in such a way that each packing is valid.

Unfortunately the number of columns of A is usually exponentially large. Thus, we start with a limited collection of these patterns, and dynamically add columns as needed. We can derive this process by applying the simplex method to this lp.

We first need an initial basic feasible solution to this problem. If we assume that $W \geq w_i, \forall i \in I$, then we can take the identity as a trivial basis. (We can assume this as otherwise the problem is trivially infeasible). Now given a basis B, we compute the simplex multipliers $p^T = c_B^T B^{-1}$. Note that $c_B = (1 \quad \cdots \quad 1)$. Now, following the framework and notation established by Bertsimas and Tsitsiklis [1], we compute the reduced costs $\overline{c}_j = 1 - p^T A_j$. In traditional simplex, we would then just compute this for each pattern, and find the minimum. If the minimum is negative, then we can add that column and pivot according to it, otherwise we reached optimum. Instead of explicitly computing this reduced cost for every pattern, we instead setup a subproblem that finds this minimum. Re-framing it as a maximization problem, we get:

$$\begin{aligned} \text{maximize} \quad & \sum_{i=1}^{m} p_i x_i \\ \text{subject to} \quad & \sum_{i=1}^{m} w_i x_i \leq W, \\ & x_i \in \{0, 1\} \qquad i = 1, \ldots, m. \end{aligned} \tag{KS}$$

This is the 0/1 Knapsack Problem, which we can then solve using QAOA or other methods.

While [7] details the theoretic mapping of 0/1 Knapsack to an instance of QAOA, we utilize the Qiskit optimization module's physical mapping. Qiskit automatically handles the conversion of the classical Knapsack instance into a QUBO, and the creation of the required Hamiltonians. The problem Hamiltonian introduced in [7] is:

$$C = C_A + C_B$$

where,

$$C_A = \sum_{i=1}^{n} \frac{v_i}{2} \left(I - \sigma_z^{(i)} \right)$$

and,

$$C_B = -k \left[W_o I - \sum_{i=1}^{n} \frac{w_i}{2} \left(I - \sigma_z^{(i)} \right) \right]^2$$

and the mixer Hamiltonian is:

$$B = -\sum_i \sigma_x^{(i)}$$

4 METHOD

We use a hybrid classical-quantum approach in our column generation. We use the Gurobi solver to handle all classical optimization, and either QAOA or a $\frac{1}{2}$-approximation algorithm for a classical baseline. The overall algorithm is as follows:

Algorithm 1 Column Generation for Bin Packing

```
Initialize the identity matrix as our initial basic feasible solution.
loop
    Solve the master problem using Gurobi.
    Compute the vector of dual costs (called p in the previous section).
    Solve the generated subproblem. (QAOA or classical)
    Compute the approximation to the minimum reduced cost.
    if the approximation is negative then
        Use the newly generated column associated with the output to do a pivot, and add that column to the list of all columns.
    end if
    if the approximation is non-negative then
        fallback to the exact solver, and compute the corresponding true minimum.
        if the true minimum is negative then
            Use the newly generated column associated with the output to do a pivot, and add that column to the list of all columns.
        end if
        if the true minimum is non-negative then
            Terminate, with optimality.
        end if
    end if
end loop
```

For the classical approximation, we use a simple $\frac{1}{2}$-approximation for Knapsack that uses the value to weight ratio of items. We follow the logic provided at [4].

As is mentioned in the above pseudo-code, we use the Gurobi optimizer for all classical optimization, and the Qiskit optimization library for the QAOA. We use the Qiskit AerSimulator, to evaluate on a noiseless and noisy simulator.

As we run this column generation, we keep track of when the fallback is triggered for each method, along with the objective values, and the reduced costs.

5 EVALUATION

To evaluate all of these methods, we took Scholl instances as specified in [5], and scaled them down.

To scale them down, we randomly sample 10 instances of these Scholl problems. Then for each instance, we randomly sample 6 items from that instance, and generate a capacity from these sampled w_is.

We calculate a new bin capacity W as follows:

$$W = \left\lfloor \frac{1}{2} \sum_i w_i \right\rfloor$$

This way the solutions are not trivialized by the downscaling.

We use a median noise model based on the ibm_rensselaer qpu for our noisy simulation, which used median depolarizing, readout, and thermal relaxation errors. We also evaluate our method on the real ibm_rensselaer qpu.

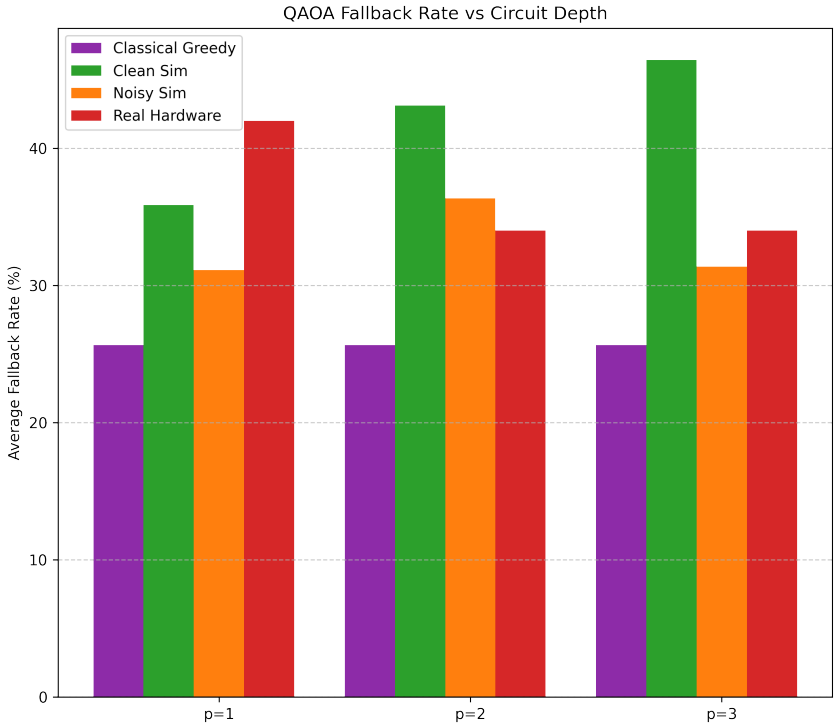
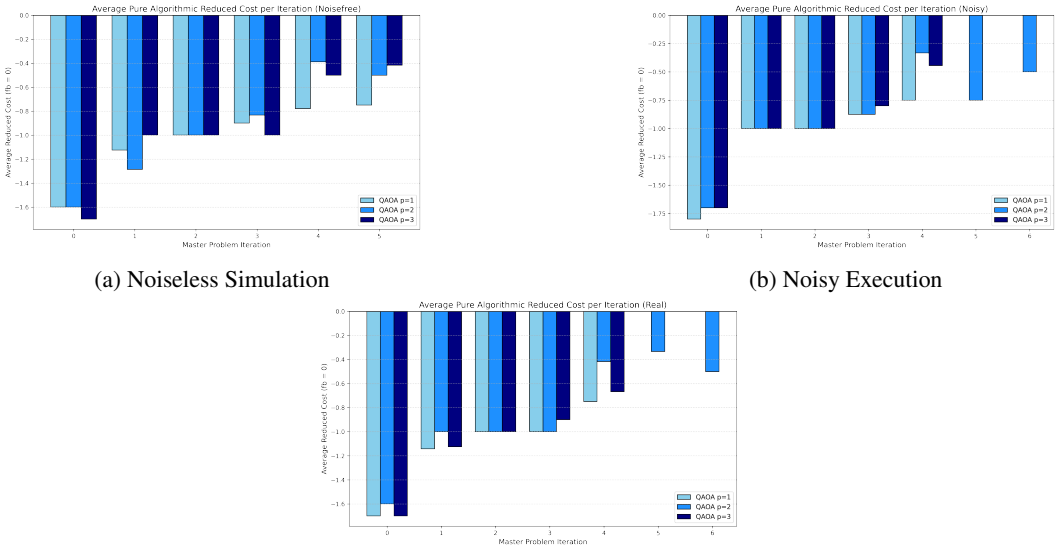


Figure 1: Average fallback rates for classical vs QAOA ($p = 1, 2, 3$). In the current NISQ era, the classical $\frac{1}{2}$-approximation outdoes QAOA. Interestingly, real hardware and noisy simulation seem to outperform the noiseless simulator.

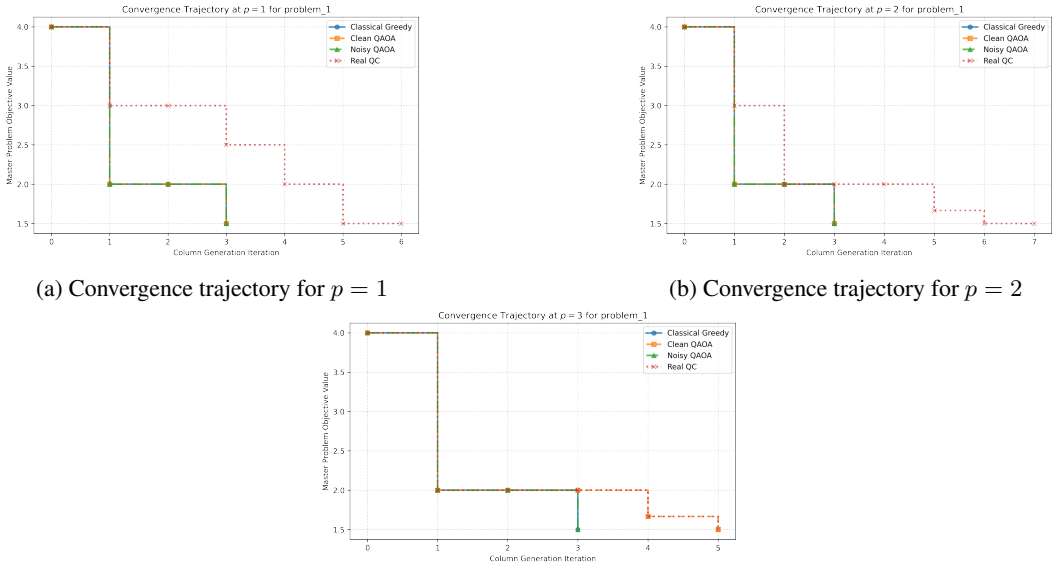


(a) Noiseless Simulation

(b) Noisy Execution

(c) Physical Hardware Execution

Figure 2: Comparison of algorithmic reduced cost progression between (a) ideal simulation, (b) noisy simulation, and (c) physical execution on the IBM Eagle processor. For smaller values of p ($p = 1, 2$), there is a noticeable slowdown in the real hardware, as compared with any other model. At larger values of p, all of these methods perform similarly according to this metric.



(a) Convergence trajectory for $p = 1$

(b) Convergence trajectory for $p = 2$

(c) Convergence trajectory for $p = 3$

Figure 3: Comparison of convergence trajectory (master problem objective values) on problem one for (a) $p = 1$, (b) $p = 2$, and (c) $p = 3$ on all knapsack solving paradigms tested. Increasing the depth parameter p provides some advantages that are heavily mitigated at later iterations by physical gate noise. Overall, $p = 1$ outperforms the other values according to this metric.

6 Conclusion

In this report, we studied the efficacy of using QAOA and column generation, in a hybrid quantum-classical approach, to solve the Bin Packing/ Cutting Stock problem. Our method involved down-

sampling the Scholl dataset, and benchmarking QAOA as a subproblem solver. By running this column generation on simulation and real hardware, we established that while current quantum devices do not outperform classical algorithms, the noise present is not a prohibitive hindrance in their performance.

Future work should focus on scaling up this hybrid architecture, running experiments on larger instances of this problem and other combinatorial problems. This can be achieved by integrating other data-reduction strategies such as coreset selection, to down-sample the number of weights the QAOA is classically trained on.

CODE AVAILABILITY

The complete codebase, including the Jupyter notebook, pre-calculated data arrays, and plotting scripts used to generate the results and figures in this report, is publicly available on GitHub at: `https://github.com/Obscurian/Quantum_Bin_Packing_QAOA`.

7 USE OF AI

I used AI as a tool to help me debug a lot of my code, to explain some of the prerequisite theory, and to generate data visualization code. I also used it to help improve the tone and style of writing in the various reports and presentations done along with this report. All of these were allowed under the class policy.

REFERENCES

[1] BERTSIMAS, D., AND TSITSIKLIS, J. *Introduction to Linear Optimization*. Athena Scientific books. Athena Scientific, 1997.

[2] COWTAN, A., DILKES, S., DUNCAN, R., KRAJENBRINK, A., SIMMONS, W., AND SIVARAJAH, S. On the qubit routing problem. *arXiv preprint arXiv:1902.08091* (2019).

[3] DA SILVA COELHO, W., HENRIET, L., AND HENRY, L.-P. Quantum pricing-based column-generation framework for hard combinatorial problems. *Physical Review A 107*, 3 (2023), 032426.

[4] GEEKSFORGEEKS. Approximation algorithms for knapsack. `https://www.geeksforgeeks.org/dsa/approximation-algorithms-for-knapsack/`, 2025. Accessed: 2026-04-28.

[5] GILMORE, P. C., AND GOMORY, R. E. A linear programming approach to the cutting-stock problem. *Operations Research 9*, 6 (1961), 849–859.

[6] GILMORE, R., AND GOMORY, R. A linear programming approach to the cutting stock problem i. *Oper Res 9* (01 1961).

[7] UR REHMAN, J., AL-HRAISHAWI, H., AND CHATZINOTAS, S. Quantum approximate optimization algorithm for knapsack resource allocation problems in communication systems. In *ICC 2023 - IEEE International Conference on Communications* (2023), pp. 2674–2679.

[8] YAO, A. C.-C. New algorithms for bin packing. *Journal of the ACM (JACM) 27*, 2 (1980), 207–227.